

Pointers – advanced topics

Lecture overview

- Pointers, arrays, and strings – recap ←
- Faulty memory access
- Pointer type conversion
- Program arguments in C
- Dynamic memory allocation

Character Arrays – recap

Example:

```
#include <stdio.h>
#define SIZE 2

int main() {
    int i;
    char initials[SIZE] = {'J', 'D'}; // initials of "John Doe"
    char *p = initials;

    for (i = 0; i < SIZE; i++)
        printf ("_%c", p[i]);
    printf ("_\n");
    return 0;
}
```

- Condition of `i < SIZE` since length of initials character array is exactly size SIZE

_J_D_

Character Arrays – recap

- For most character arrays, though, length is not exactly SIZE, e.g., full name can be 8 characters long or 25 character long
- Therefore, we use '\0' to determine the boundary
 - `printf("%s")` and `strlen` rely on '\0' to determine the boundary
 - If there is no null terminator, then their output is undetermined

Arrays, pointers and strings – recap

Example:

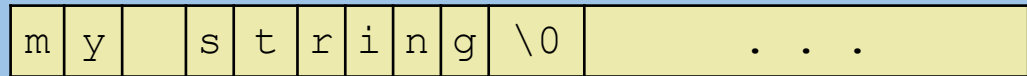
```

#include <stdio.h>
#define SIZE 20

int main() {
    int i;
    char str[SIZE] = "my string";

    for (i = 0; i < SIZE && str[i] != '\0'; i++)
        printf ("_%c", str[i]);
    printf ("_\\n");
    return 0;
}

```



- Array `str` has 20 entries.
 - The first 9 hold characters of "my string"
 - The remaining 11 hold the (ASCII) value 0 ('\0')

_m_y_ _s_t_r_i_n_g_

Arrays, pointers and strings – recap

Example:

```
#include <stdio.h>
#define SIZE 20

int main() {
    int i;
    char str[SIZE] = "my string";
    char *p = str;

    for (i = 0; i < SIZE && str[i] != '\0'; i++)
        printf ("_%c", p[i]);
    printf ("_\\n");
    return 0;
}
```

- Entries of **array** **str** are accessed using the **free pointer** **p**.
 - The indexing operator (**[]**) can be used on arrays (such as **str**) and on pointer variables (such as **p**)

_m_y_ _s_t_r_i_n_g_

Arrays, pointers and strings – recap

Example:

```
#include <stdio.h>
#define SIZE 20

int main() {
    int i;
    char str[SIZE] = "my string";
    char *p = str;

    for (i = 0; i < SIZE && str[i] != '\0'; i++)
        printf ("_%c", *(p+i));    // *(p+i) is the same as p[i]
    printf ("\n");
    return 0;
}
```

- Entries of **array str** are accessed using the **free pointer p**.
 - The indexing operator performs **de-referencing with an offset**

_m_y_ _s_t_r_i_n_g_

Pointer arithmetic operations

- `ptr` is an address of a certain pointer **type** (e.g. , `int*` or `double*`)
- `*ptr` is some integer (positive or negative)
- `ptr + i` is the address held by `ptr + ( i x sizeof(type) )`

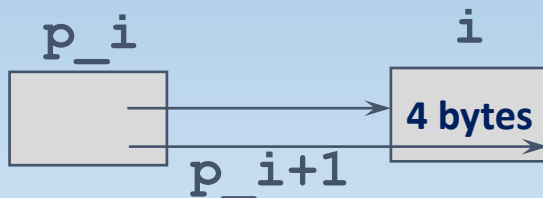
Example:

```

int    i = 7,
      *p_i = &i;

printf("p_i=%p, p_i+1=%p\n", p_i, p_i+1);
    
```

`p_i=0xbfffffff4d4`, `p_i+1=0xbfffffff4d8` ←address difference of 4



Pointer arithmetic operations

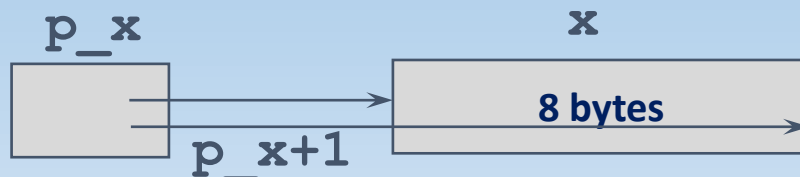
- `ptr` is an address of a certain pointer **type** (e.g. , `int*` or `double*`)
- `*ptr` is some integer (positive or negative)
- `ptr + i` is the address held by `ptr + ( i x sizeof(type) )`

Example:

```
double x = 7.0,
      *p_x = &x;

printf("p_x=%p, p_x+1=%p\n", p_x, p_x+1);
```

`p_x=0xbfffffff4c0`, `p_x+1=0xbfffffff4c8` ← address difference of 8



Arrays, pointers and strings – recap

Example:

```
#include <stdio.h>
#define SIZE 30

int main() {
    int i;
    char str[SIZE] = "my string";
    char *p = str;

    for (i = 0; i < SIZE && str[i] != '\0'; i++)
        printf ("_%c", p[i]);    // *(p+i) is the same as p[i]
    printf ("_\n");
    return 0;
}
```

- Entries of **array str** are accessed using the **free pointer p**
 - The indexing operator performs **de-referencing with an offset**

_m_y_ _s_t_r_i_n_g_

Arrays, pointers and strings – recap

Example:

```
#include <stdio.h>
#define SIZE 30

int main() {
    int i;
    char str[SIZE] = "my string";
    char *p = str;

    for (i = 0; i < SIZE && str[i] != '\0'; i++)
        printf ("_%c", *(p+i));    // *(p+i) is the same as p[i]
    printf ("_\n");
    return 0;
}
```

- Entries of **array str** are accessed using the **free pointer p**
 - The indexing operator performs **de-referencing with an offset**
 - The de-referencing (*) and indexing ([]) operators can both be applied to array and pointer variables

_m_y_ _s_t_r_i_n_g_

Arrays, pointers and strings – recap

Example:

```
#include <stdio.h>
#define SIZE 30

int main() {
    int i;
    char str[SIZE] = "my string";
    char *p = str;

    for (i = 0; i < SIZE && str[i] != '\0'; i++)
        printf ("_%c", *(p++));    // p is de-referenced and advanced
    printf ("_\n");
    return 0;
}
```

- Entries of **array str** are accessed using the **free pointer p**
 - The free pointer **p** moves across the string

_m_y_ _s_t_r_i_n_g_

Arrays, pointers and strings – recap

Example:

```
#include <stdio.h>
#define SIZE 30

int main() {
    int i;
    char str[SIZE] = "my string";
    char *p = str;

    while(*p) // equivalent to: while (*p != 0), i.e., while (*p != '\0')
        printf ("_%c", *(p++));
    printf ("_\n");
    return 0;
}
```

- Entries of **array str** are accessed using the **free pointer p**
 - The free pointer **p** moves across the string
 - Traverses the string until **p** reaches terminating '\0'

_m_y_ _s_t_r_i_n_g_

Arrays, pointers and strings – recap

- Array variables are of pointer type
- Array indexing is implemented via pointer arithmetic
$$\text{arr}[i] \equiv *(\text{arr}+i)$$
(one reason why pointer type matters – we'll see more today)
- Free pointers are pointer variables not associated with an array definition
- Both array and free pointers can be de-referenced using `*` or `[]`
(both operators can actually be applied to any **value** of pointer type)
- **Function parameters are always free pointers**
(even if `[]` notation is used in type of parameter)
- There are two main differences between array variables and free pointers (see next two slides)

Arrays vs. free pointers

Difference between arrays and free pointers:

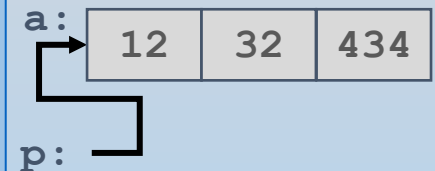
1. The **sizeof** operator can be used to obtain the size of an array (in bytes) if applied to an array variable, but not to a “free pointer” pointing to the same address

Example:

```

int a[] = {12, 32, 434};
int *p = a;
int i1 = sizeof(a), i2 = sizeof(p);

printf ("sizeof(a)=%d\nsizeof(p)=%d\n", i1, i2);
  
```



```

sizeof(a)=12    ← 3 x sizeof(int)
sizeof(p)=8     ← sizeof(int*)
  
```

- Pointer size is the same for all types, i.e., `sizeof(int*) == sizeof(double*) == sizeof(char*)`, etc. – 8 on current server
- If `arr` is an array variable, then `sizeof(arr)/sizeof(arr[0])` is the array length

Arrays vs. free pointers

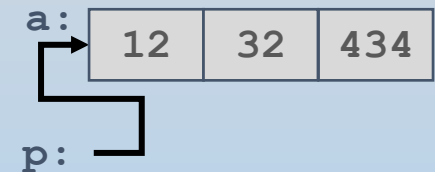
Difference between arrays and free pointers:

2. A free pointer can have its value changed, but an array variable cannot (always points to the “physical” array)

Example:

```
int a[] = {12, 32, 434};  
int *p = a, i;  
  
for (i=0; i<sizeof(a)/sizeof(a[0]); i++)  
    printf("a[i] = %d\n", *(p++));
```

`a++;` ← will trigger a compilation error



Lecture overview

- Pointers, arrays, and strings – recap
- Faulty memory access ←
- Pointer type conversion
- Program arguments in C
- Dynamic memory allocation

Faulty memory access

- Pointers in C provide a framework for accessing arbitrary addresses in the binary program
- It is possible and quite easy to abuse this framework and get into trouble ...
- **Inappropriate memory access is one of the most common causes of runtime errors in C**

Faulty memory access – example 1

```
#include <stdio.h>

int main() {
    int array[] = {1,2,3,4};

    printf("array[-1] = %d\n",array[-1]);
    printf("array[4] = %d\n",array[4]);
    printf("array[40] = %d\n",array[40]);
    return 0;
}
```

- Array bounds are not “guarded” in compile or run time
- Program can access any address relative to the beginning of the array (even a negative index)
- It is the programmer’s responsibility to ensure array bounds are not crossed!

Faulty memory access – example 2

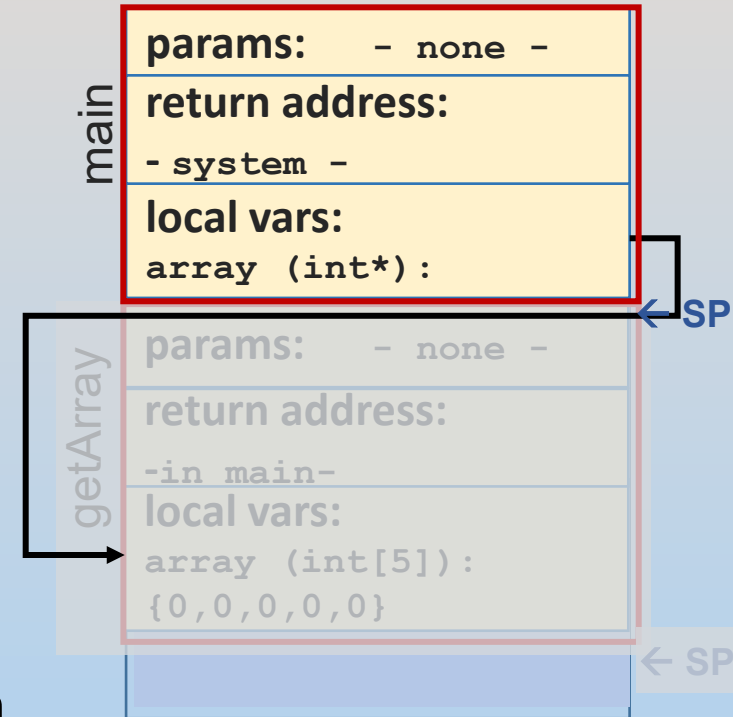
```

#include <stdio.h>

int* getArray(void) {
    int array[5] = {0};
    return array;
}

int main() {
    int *array = getArray();
    printf("array[0] = %d\n", array[0]);
    return 0;
}
  
```

- Once function returns, stack frame is freed and local array can be overwritten
- Function should not return **a pointer to local address**
- This triggers severe compilation warning



Faulty memory access – example 3

```
#include <stdio.h>

int main() {
    int *p = NULL;    // NULL is address 0
    printf("*p = %d\n", *p);
    return 0;
}
```

- Trying to de-ref an unmapped address (0) leads to “segmentation fault” during run time
- Seg fault always triggered by de-ref (* or []) and it is always during runtime

Faulty memory access – example 3

```
#include <stdio.h>

int main() {
    int *p;                // uninitialized pointer
    printf("*p = %d\n", *p);
    return 0;
}
```

- Trying to de-ref an unmapped address (0) leads to “segmentation fault” during run time
- Seg fault always triggered by de-ref (* or []) and it is always during runtime
- Often caused due to de-ref of uninitialized pointer (even if we don't set pointer to NULL)

Lecture overview

- Pointers, arrays, and strings – recap
- Faulty memory access
- **Pointer type conversion ←**
- Program arguments in C
- Dynamic memory allocation

Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

```
char    string[15] = "Hello world";  
short*  shortP;  
int*    intP;  
double* doubleP;
```

string →

'H'	'e'	'l'	'l'	'o'	' '	'w'	'o'	'r'	'l'	'd'	'\0'
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------

Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

```

char    string[15] = "Hello world";
short*  shortP;
int*    intP;
double* doubleP;

printf("First char = '%c', second char = '%c'\n", string[0], string[1]);
  
```

First char = 'H', second char = 'e'

string[0] string[1]
 string →

'H'	'e'	'l'	'l'	'o'	' '	'w'	'o'	'r'	'l'	'd'	'\0'
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------

Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

```

char    string[15] = "Hello world";
short*  shortP;
int*    intP;
double* doubleP;

printf("First char = %d , second char = %d \n", string[0], string[1]);
    
```

- print ASCII code of character

```
First char = 72 , second char = 101
```

string[0] string[1]

	72	101	108	108	111	32	119	111	114	108	100	0
string →	'H'	'e'	'l'	'l'	'o'	' '	'w'	'o'	'r'	'l'	'd'	'\0'

Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

```

char    string[15] = "Hello world";
short*  shortP;
int*    intP;
double* doubleP;

shortP = string;
printf("First short = %d, second short = %d \n", shortP[0], shortP[1]);
    
```

- pointer types do not freely convert, so assignment will trigger warning

```

warning: assignment from incompatible pointer type [-Wincompatible-pointer-types]
    shortP = string;
        ^
    
```

	72	101	108	108	111	32	119	111	114	108	100	0
string →	'H'	'e'	'l'	'l'	'o'	' '	'w'	'o'	'r'	'l'	'd'	'\0'

Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

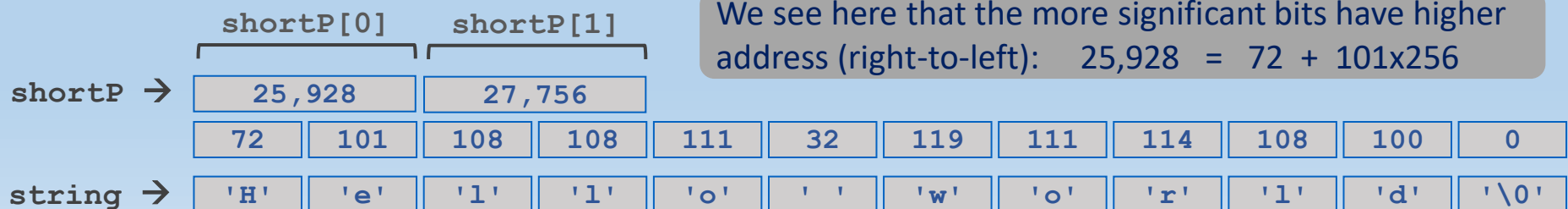
```

char    string[15] = "Hello world";
short*  shortP;
int*    intP;
double* doubleP;

shortP = string;
printf("First short = %d, second short = %d \n", shortP[0], shortP[1]);
    
```

- pointer types do not freely convert, so assignment will trigger warning

First short = 25928, second short = 27756



We see here that the more significant bits have higher address (right-to-left): $25,928 = 72 + 101 \times 256$

Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

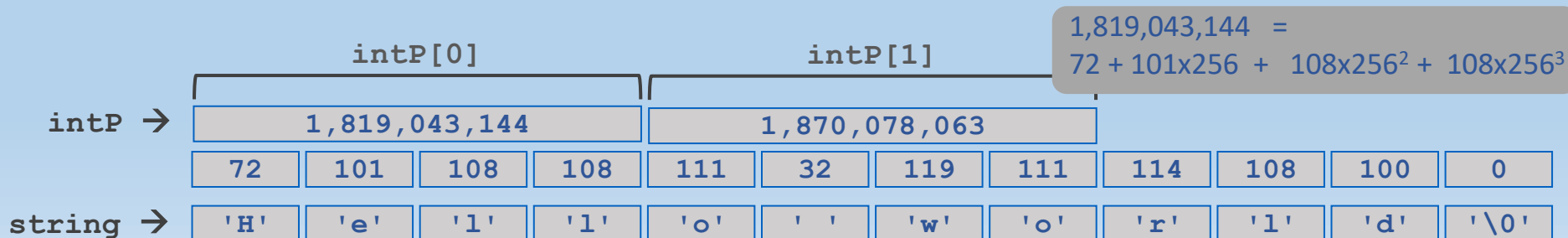
```

char    string[15] = "Hello world";
short*  shortP;
int*    intP;
double* doubleP;

intP = (int*)string;
printf("First int = %d, second int = %d \n", intP[0], intP[1]);
    
```

- pointer types convert without warning when using casting operator

First int = 1819043144, second int = 1870078063



Pointer type conversions

- The pointer type (e.g., `int*` or `double*`) contains information about the **number of bytes** being referenced and how to **interpret their content**

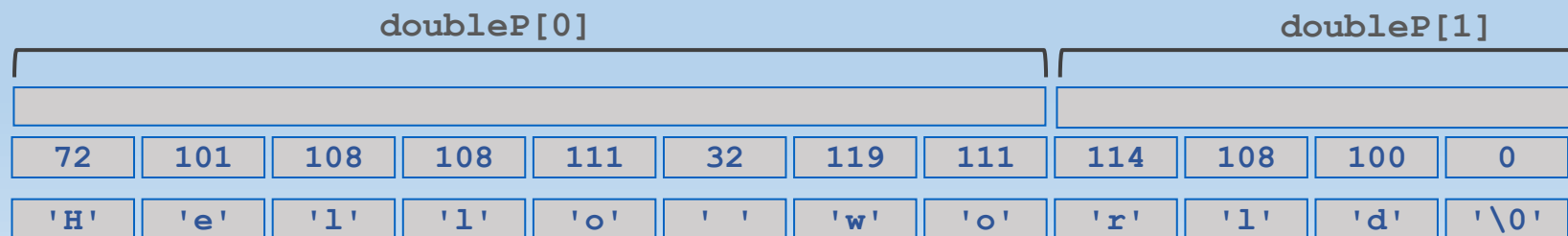
```
char    string[15] = "Hello world";
short*  shortP;
int*    intP;
double* doubleP;
```

```
doubleP = (double*)string;
printf("First double = %g, second double = %g\n", doubleP[0], doubleP[1]);
```

%g for printing in scientific notation

- Converting to double will convert “meaning” of stored bits

First double = 8.76578e+228, second double = 3.25162e-317



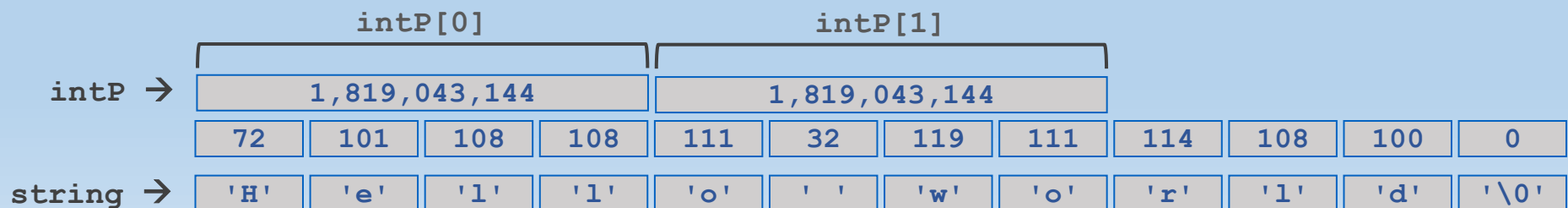
Pointer type conversions

Summary:

- Pointer type conversion should be avoided
- If you really wish to covert a pointer to a different type, use casting to avoid compilation warnings
- If you wish to convert the type of the referenced value, apply conversion after de-referencing

```
char    string[15] = "Hello world";
double  d = *string;
printf("d = %.11f\n", d);
```

d = 72.0



Homework Exercise #5 Insights

- **Not all spaces are visible** in SystemProg-ex5.pdf
 - Sometimes output contain spaces that are hard to see – don't rely on visual alignment
 - Instead, use the **provided sample executables** to validate results:
`/share/ex_data/ex5/abcNewCode`
- **Visualization vs. encoded output**
 - Examples in SystemProg-ex5.pdf may **visualize** results for clarity (e.g., " BC")
 - The actual **encoded result** omits leading zeros (" BC" → 11 → "BC"), and uses 'x' to represent spaces
- **Reuse your implemented functions**
 - Don't reimplement logic – leverage the encoding functions you've already written
- **Modify only where instructed**
 - Stick to the guidelines – changing out of the specified areas may break test validation
 - Tests are strict and may not point to the exact line of error

Lecture overview

- Pointers, arrays, and strings – recap
- Faulty memory access
- Pointer type conversion
- Program arguments in C ←
- Dynamic memory allocation

Command line arguments

Until now we referred to the prototype of `main` as follows:

```
int main () ;
```

but its (more) complete prototype is:

```
int main (int argc, char *argv[]) ;
```

or

```
int main (int argc, char **argv) ;
```

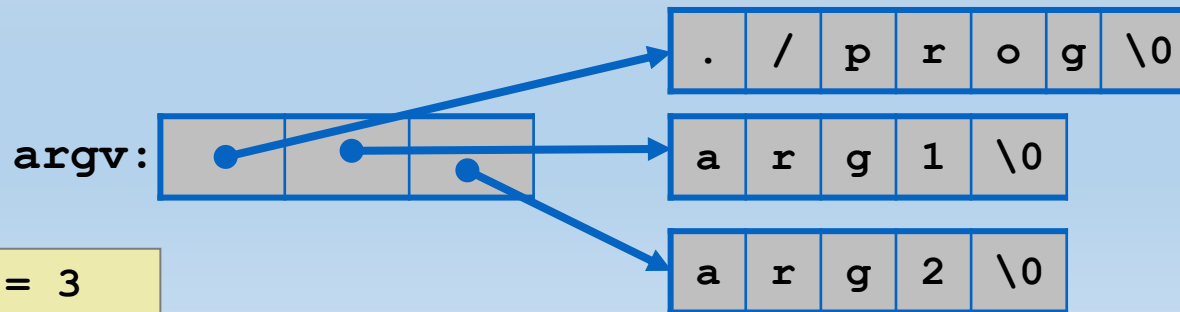
- `argc` stands for *argument count* – the number of arguments
- `argv` stands for *argument vector* – an array of strings

Command line arguments

main() argument **argv** is of type **char**** – an array of pointers, pointing to the starting points of strings (allocated in stack frame of **main**)

- **argv[0]** (type **char***) is the string holding the program's name
- **argv[i]** (type **char***) ($i > 0$) is the string holding argument number **i** (of actual arguments)
- **argv[i][j]** (type **char**) ($i > 0, j \geq 0$) is the $(j+1)^{\text{th}}$ character of **argv[i]**

```
==> ./prog arg1 arg2
```



```
argc = 3
```

Command line arguments – example

A simple program that prints its command line arguments:

```
#include <stdio.h>

int main (int argc, char *argv[]) {
    int    i;

    printf("There are %d elements in the array:\n", argc);
    for(i=0; i<argc; i++) {
        printf("argv[%d] = %s\n",i, argv[i]);
    }
    return 0;
}
```

code in /share/classes/class09/printArgs.c

```
==>./printArgs hello
There are 2 elements in the array:
argv[0] = ./printArgs    ←first argument is program name
argv[1] = hello

==>./printArgs "hello world" how are you
There are 5 elements in the array:
argv[0] = ./printArgs
argv[1] = hello world    ←argument with space wrapped in
argv[2] = how             quote marks ("...")
argv[3] = are
argv[4] = you
```

Command line arguments

Final important note:

- The arguments are always read as strings. If you wish to convert to other type (**int** or **double**) you need to invoke some conversion function

```

int my_atoi (const char* str) {
    int num=0, sign=1;
    char c = *str;
    if(c=='-') {
        str++;
        sign = -1;
    } else if(c=='+') {
        str++;
    }

    while(c=*str) {
        if(c<'0' || c>'9')
            return 0;
        num = num*10 + (c-'0');
        str++;
    }
    return num*sign;
}
    
```

Compute sign (-1 or +1)

← while value isn't '\0'
 ← if c contains a non-digit char, return 0
 ← otherwise, iteratively update num and increment address held by pointer str
 ← return sign * absolute value

Compute absolute value

atoi() (stands for array-to-int) is also a standard library function in **stdlib**.

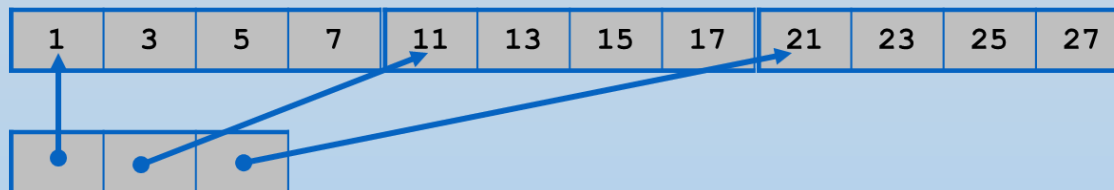
Multidimensional arrays in C

There are two ways to implement 2D arrays in C:

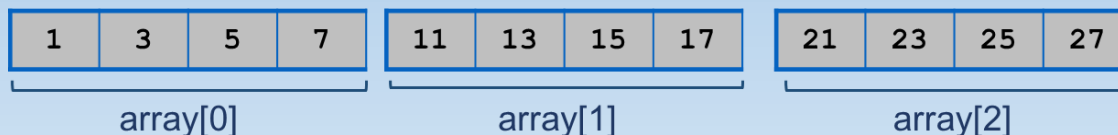
1	3	5	7
11	13	15	17
21	23	25	27

We cover this in detail in the last lecture (in 3 weeks . . .)

- An array of pointers to row starts (e.g., type `int**`)
[this is how `argv` implements an array of strings]



- An array of built-in arrays (e.g., type `int[][4]`)
[the length of the row arrays is part of the type of the 2D array]



Lecture overview

- Pointers, arrays, and strings – recap
- Faulty memory access
- Pointer type conversion
- Program arguments in C
- **Dynamic memory allocation ←**

Dynamic memory allocation

All memory allocation we discussed until now (local and global variables) was determined during compilation

- Space for global variables is “hard coded” in the **data segment**
- Space for local variables is **automatically** allocated when a function’s **stack frame** is loaded to the stack

Dynamic memory allocation

All memory allocation we discussed until now (local and global variables) was determined during compilation

- Space for global variables is “hard coded” in the **data segment**
- Space for local variables is **automatically** allocated when a function’s **stack frame** is loaded to the stack

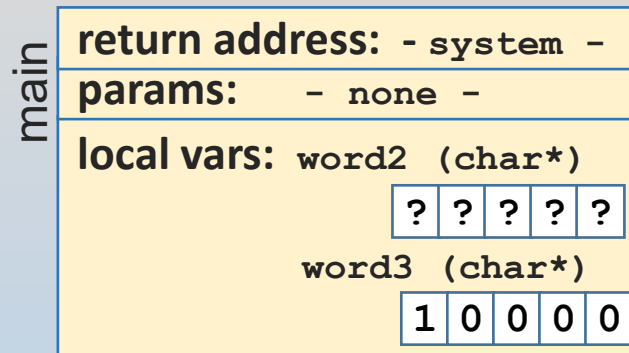
What if we want our program to use storage that will be determined at runtime?

We need to use additional space not on stack or data segment – rather use space on **the heap**

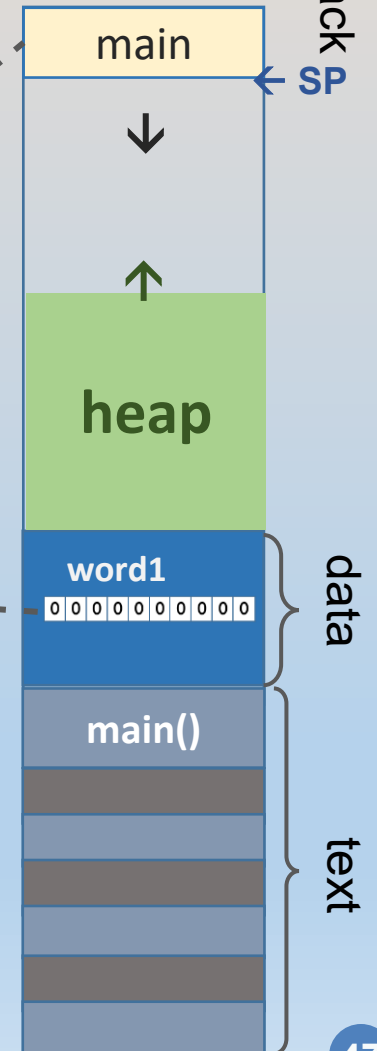
Array allocation example

```
char word1[10];

int main() {
    char word2[5];
    char word3[5]={1};
    ...
    return 0;
}
```



0 0 0 0 0 0 0 0 0 0



- The heap is a block of memory that sits between the data segment and the stack
- It typically “grows” upwards toward the stack (configuration is system dependent)

Dynamic memory allocation

How does it work?

- Allocate continuous block of memory using a memory allocation library function (`malloc`, `calloc`, `realloc`)
- Memory block can be used globally in the program, because it does not get automatically freed
- Free block of memory by calling a memory freeing library function (such as `free()`)

➔ Relevant system library: `stdlib.h`

Dynamic memory allocation – example 1

Using a local array (automatically allocated on the stack):

```
#include <stdio.h>

int main(int argc, char *argv[]) {

    float arr[20];           ← Local array allocated on the stack

    arr[0] = 2;
    printf ("Value of first cell: %f\n", arr[0]);
    arr[7] = 17;
    printf ("Value of 8th cell:    %f\n", arr[7]));

    return 0;
}
```

Dynamic memory allocation – example 1

Using a dynamically allocated array:

```
#include <stdio.h>

int main(int argc, char *argv[]) {

    float *arr;                                ← Free pointer that should point to allocated space

    int numElements;
    if(argc < 2) return 1;
    numElements = atoi(argv[1]);
    if(numElements == 0) return 1;             } ← Read number of elements from program argument

    arr[0] = 2;                                ← Will likely lead to segmentation fault
    printf ("Value of first cell: %f\n", arr[0]);
    arr[7] = 17;
    printf ("Value of 8th cell:    %f\n", arr[7]));

    return 0;
}
```

Dynamic memory allocation – example 1

Using a dynamically allocated array:

```

#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {

    float *arr;

    int numElements;
    if(argc < 2)          return 1;
    numElements = atoi(argv[1]);
    if(numElements == 0) return 1;

    arr = malloc (numElements * sizeof (float));

    arr[0] = 2;
    printf ("Value of first cell: %f\n", arr[0]);
    arr[7] = 17;
    printf ("Value of 8th cell:   %f\n", arr[7]));

    free(arr);
    return 0;
}
  
```

← Include **stdlib**

← Free pointer that should point to allocated space

← Read number of elements from program argument

← Allocate space

← Use space

← Free space

malloc

```
void* malloc(size_t)
```

- Input: number of **bytes** to allocate
(`size_t` is an integral type returned by `sizeof`)
- Output:
 - If successful – start address (`void*`) to the memory block allocated
 - If failed – returns NULL **← always check!!**
(recall: NULL is 0 in pointer type)

Generic pointer

- Type `void*` (generic pointer) is used to hold an address without the context of a specific type
- Conversion of any pointer type **to** / **from** `void*` is done without warning

```
int i=1;
void* p_v = &i;
int* p_i = p_v;
double* p_d = p_v;
```



Generic pointer

- Type `void*` (generic pointer) is used to hold an address without the context of a specific type
- Conversion of any pointer type **to** / **from** `void*` is done without warning
- A generic pointer cannot be de-referenced (no type context!)

```
int i=1;
void* p_v = &i;
int* p_i = p_v;
double* p_d = p_v;
printf("i = %d, *p_i = %d, *p_v = %d\n", i, *p_i, *p_v);
```

Triggers warning and error:

```
warning: dereferencing 'void *' pointer
      ^
error: invalid use of void expression
```

Dynamic memory allocation – example 1

Using a dynamically allocated array:

```
int main(int argc, char *argv[]) {  
  
    float *arr;  
    int numElements;  
    . . .  
  
    arr = malloc (numElements * sizeof (float));    ← Allocate space  
  
    arr[0] = 2;                                     ← Use space  
    . . .  
  
    free(arr);                                       ← Free space  
    return 0;  
}
```

- Automatic conversion of **void*** to **float*** (no need for casting)
- De-reference pointer (array index) only after conversion

Dynamic memory allocation – example 1

Using a dynamically allocated array:

```
int main(int argc, char *argv[]) {  
  
    float *arr;  
    int numElements;  
    . . .  
  
    arr = malloc (numElements * sizeof (float));  
    if(!arr) {  
        printf("Out of memory!\n");  
        return 1;  
    }  
    arr[0] = 2;  
    . . .  
  
    free(arr);  
    return 0;  
}
```

← Allocate space
+ validate allocation

← Use space

← Free space

- Automatic conversion of **void*** to **float*** (no need for casting)
- De-reference pointer (array index) only after conversion
- Do not forget to validate allocation (since it will cause seg-fault, if allocation had failed)

free

```
void free(void*)
```

- Input: the address of a previously-allocated block of memory
- Freeing memory **does not flag the contents as “freed”**; it just changes the program’s memory allocation tables to indicate this block can be allocated later
- **Common mistakes** (resulting in runtime error):
 - freeing the same block more than once
 - using free on a pointer to an address that has not been allocated
 - **using a pointer after it has been freed**
(will not actually trigger an error message!)

Dynamic memory allocation – example 2

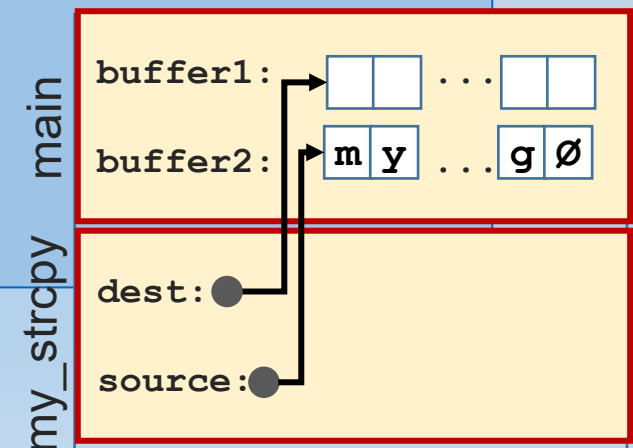
Function `strcpy` copies string into a pre-allocated buffer:

```

int main() {
    char buffer1[20],           // dest
          buffer2[] = "my string"; // source
    my_strcpy(buffer1, buffer2);
    . . .
}
  
```

```

char* my_strcpy(char* dest, const char* source) {
    int i;
    for(i=0; source[i]; i++)
        dest[i] = source[i];
    dest[i] = '\0';
    return dest;
}
  
```



Dynamic memory allocation – example 2

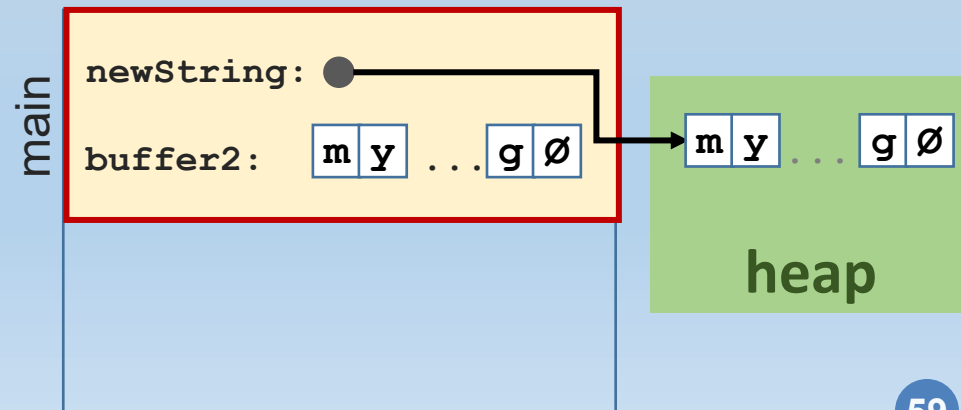
Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

```

int main() {
    char *newString,  buffer2[]  = "my string";
    newString = strdup(buffer2);
    . . .

    free(newString);
    . . .
}
  
```

- We allocate the exact amount of space we need for string
- We use it throughout our program
- We don't forget to free it when we're done



Dynamic memory allocation – example 2

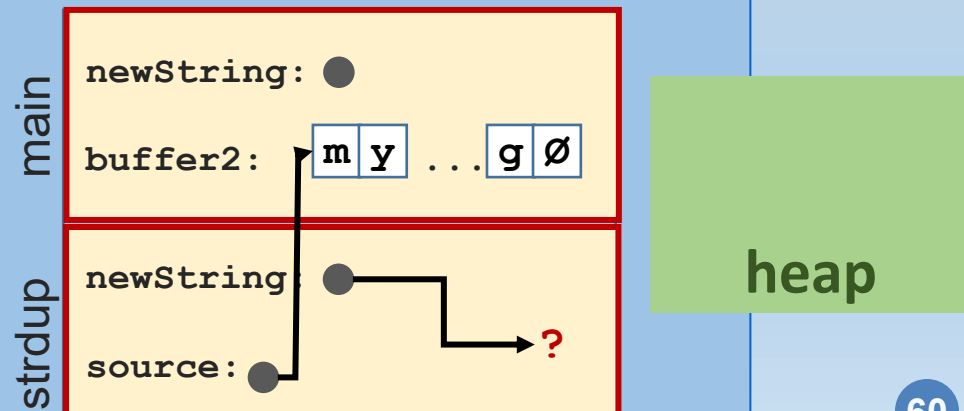
Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

```
int main() {
    char *newString, buffer2[] = "my string";
    newString = strdup(buffer2);

    . . .
    free(newString);
    . . .
}
```

```
char* strdup(const char* source) {
    char* newString;

    . . .
}
```



Dynamic memory allocation – example 2

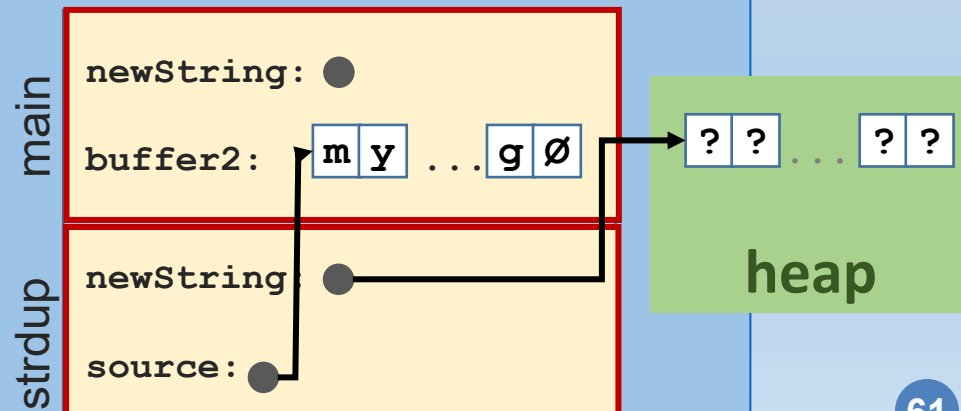
Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

```
int main() {
    char *newString, buffer2[] = "my string";
    newString = strdup(buffer2);

    . . .
    free(newString);
    . . .
}
```

```
char* strdup(const char* source) {
    char* newString;
    newString =
        malloc(1+strlen(source));

}
```

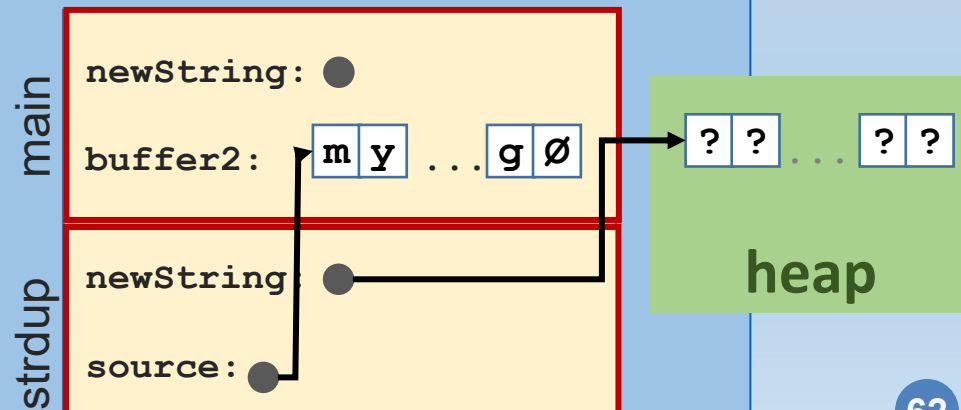


Dynamic memory allocation – example 2

Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

```
int main() {
    char *newString, buffer2[] = "my string";
    newString = strdup(buffer2);
    if(!newString) return 1;
    . . .
    free(newString);
    . . .
}
```

```
char* strdup(const char* source) {
    char* newString;
    newString =
        malloc(1+strlen(source));
    if(!newString) return NULL;
    . . .
}
```

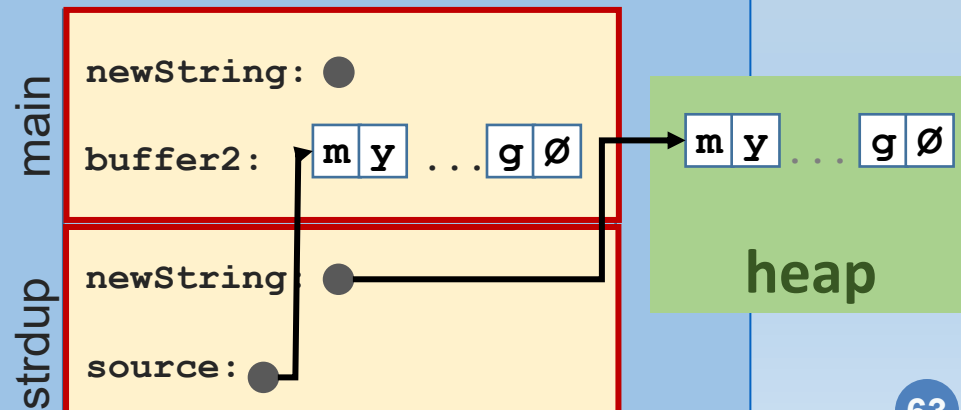


Dynamic memory allocation – example 2

Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

```
int main() {
    char *newString, buffer2[] = "my string";
    newString = strdup(buffer2);
    if(!newString) return 1;
    . . .
    free(newString);
    . . .
}
```

```
char* strdup(const char* source) {
    char* newString;
    newString =
        malloc(1+strlen(source));
    if(!newString) return NULL;
    my_strcpy(newString, source);
}
```

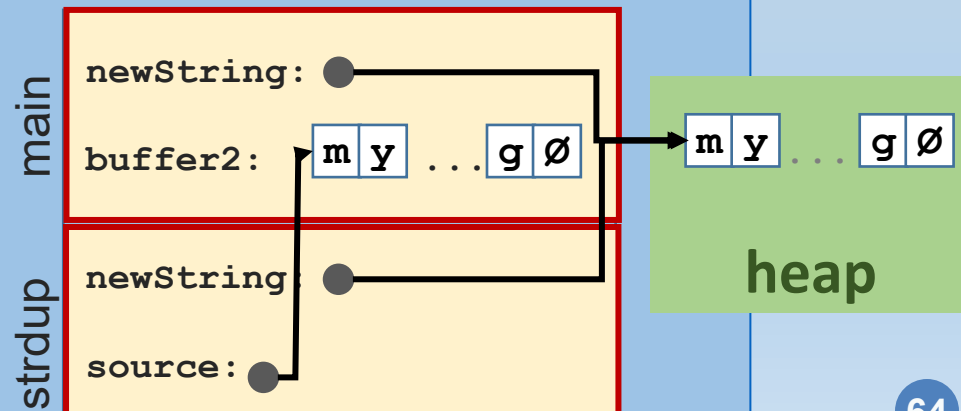


Dynamic memory allocation – example 2

Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

```
int main() {
    char *newString, buffer2[] = "my string";
    newString = strdup(buffer2);
    if(!newString) return 1;
    . . .
    free(newString);
    . . .
}
```

```
char* strdup(const char* source) {
    char* newString;
    newString =
        malloc(1+strlen(source));
    if(!newString) return NULL;
    my_strcpy(newString, source);
    return newString;
}
```



Dynamic memory allocation – example 2

Let's write a function `strdup` that duplicates a string in a newly allocated space (on the heap):

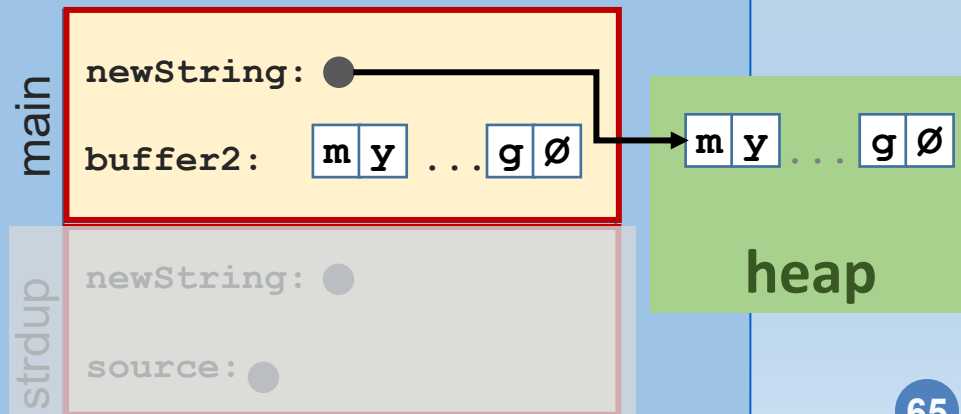
```
int main() {
    char *newString, buffer2[] = "my string";
    newString = strdup(buffer2);
    if(!newString) return 1;
    . . .
    free(newString);
    . . .
}
```

We'll see a program using this function in this week's recitation and in next lecture.

```
char* strdup(const char* source) {

    char* newString;
    newString =
        malloc(1+strlen(source));
    if(!newString) return NULL;
    strcpy(newString, source);
    return newString;

}
```



Other memory allocation functions

```
void* calloc(size_t num_elements, size_t element_size)
```

- allocates a block of size `num_elements` x `element_size` and initialize the allocated memory to all-0

```
void* realloc(void*, size_t)
```

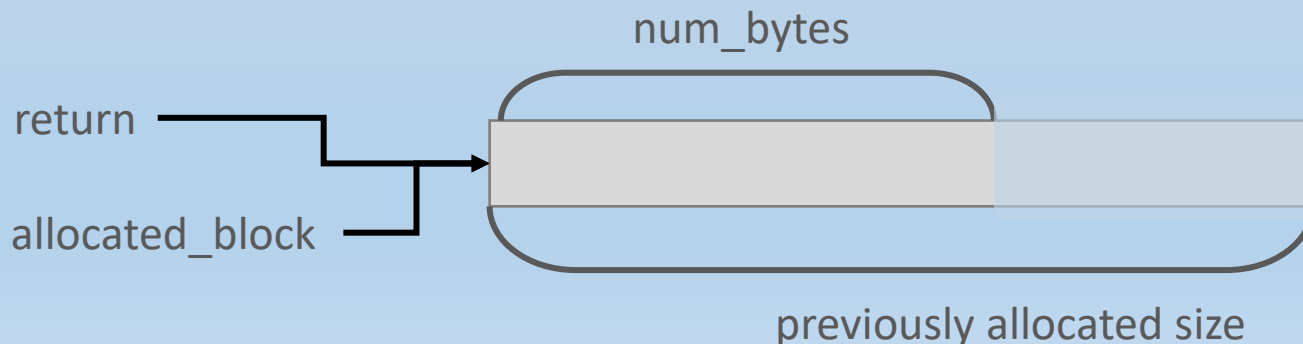
- Changes the size (expand or shrink) of a **previously allocated block**. Returns the address of new block
- Will return a different address from one specified only if there is no space to expand in current position
- Given a NULL pointer, `realloc()` behaves like `malloc()`

Additional memory allocation functions

We use `malloc()` for most allocation purposes, but `stdlib` contains several other function that can be useful in some cases:

```
void* realloc(void* allocated_block, size_t num_bytes)
```

Scenario 1: If `allocated_block` is **larger** than `num_bytes`, then `realloc` simply records that `allocated` is now of size `num_bytes`. This effectively frees the end of the block



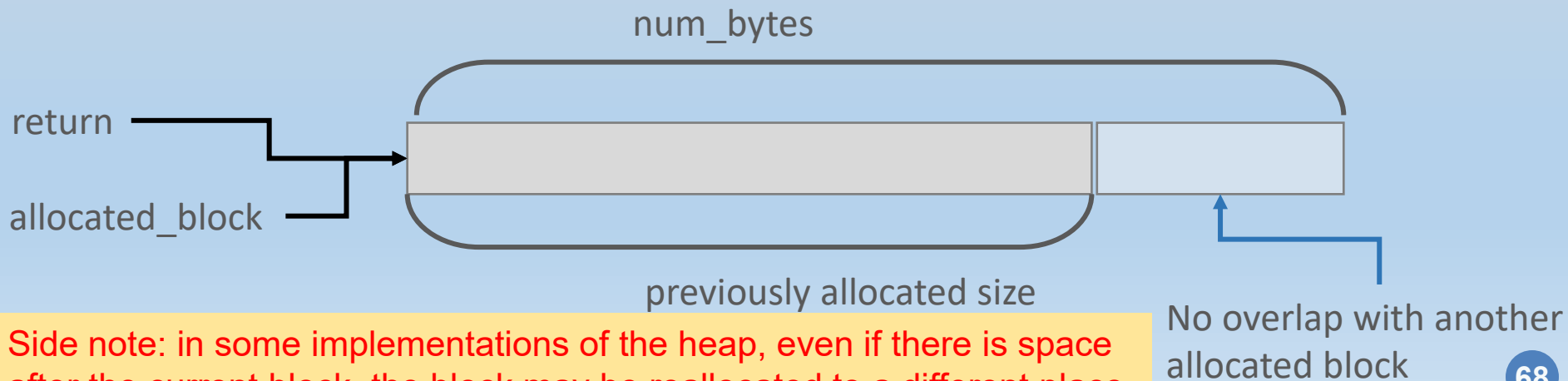
Side note: in some implementations of the heap, even here, the block may be reallocated to a different place

Additional memory allocation functions

We use `malloc()` for most allocation purposes, but `stdlib` contains several other function that can be useful in some cases:

```
void* realloc(void* allocated_block, size_t num_bytes)
```

Scenario 2: If `allocated_block` is **smaller** than `num_bytes`, but the block can be extended forward without overlapping with another allocated block, then **`realloc`** simply records that the allocated block is now of size `num_bytes`. This effectively extends the same allocation



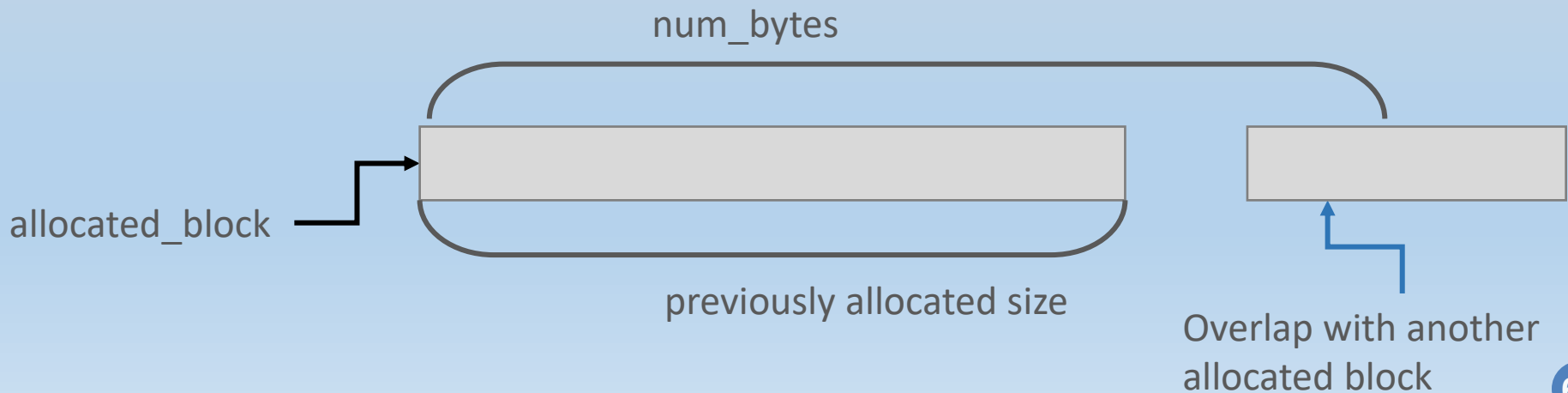
Side note: in some implementations of the heap, even if there is space after the current block, the block may be reallocated to a different place

Additional memory allocation functions

We use `malloc()` for most allocation purposes, but `stdlib` contains several other function that can be useful in some cases:

```
void* realloc(void* allocated_block, size_t num_bytes)
```

Scenario 3: If `allocated_block` is **smaller** than `num_bytes`, but extending the block forward will overlap with another allocated block, then:



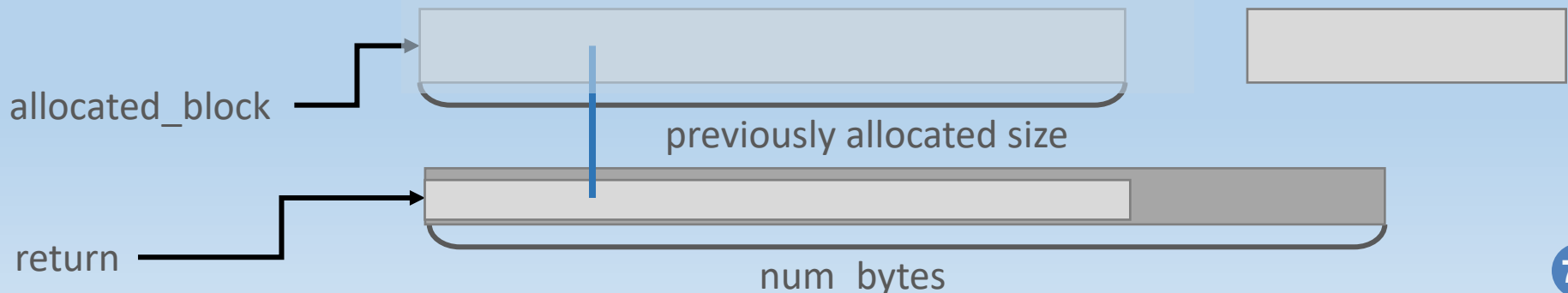
Additional memory allocation functions

We use `malloc()` for most allocation purposes, but `stdlib` contains several other function that can be useful in some cases:

```
void* realloc(void* allocated_block, size_t num_bytes)
```

Scenario 3: If `allocated_block` is **smaller** than `num_bytes`, but extending the block forward will overlap with another allocated block, then `realloc` function:

- allocates a new block of the required size
- copies all content of the original block into the beginning of the newly allocated block
- frees the old block and returns the address of the new block



Dynamic memory maintenance

- Dynamically allocated memory is given from the heap, and it is **globally accessible** across all functions of the program
- Thus, you may **allocate** memory in one function and **free** it in another function
- It is the programmer's **responsibility to free memory**

Dynamic memory maintenance

- Dynamically allocated memory is given from the heap, and it is **globally accessible** across all functions of the program
- Thus, you may **allocate** memory in one function and **free** it in another function
- It is the programmer's **responsibility to free memory**
- **Memory leaks** happen when the program no longer holds a pointer to an allocated memory block, which as a consequence cannot be freed
- Avoiding **memory leaks** requires a lot of attention and care when coding
- We will see specific examples of memory leaks **next week**

At home...

- Homework exercise #4 was due **Sunday, May 31 @ 21:00**
- Homework exercise #5 published, due **Sunday, June 14 @ 21:00**
- Review examples on dynamic memory allocation before next lecture